Laporan Tugas Besar 2 IF2211 Strategi Algoritma

Semester II tahun 2025/2026

Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada pohon Document Object Model

Oleh

Billie Bhaskara Wibawa 13524024
Raysha Erviandika Putra 13524050
Dzaki Ahmad Al Hussainy 13524084

Program Studi Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung 2025/2026

Daftar Pustaka

Daftar Pustaka.....1
1. BAB 1 Deskripsi.....2
1.1. Latar Belakang.....2
1.2. Tugas.....2
2. BAB 2 Landasan Teori.....3
2.1. Struktur Dasar Web.....3
2.1.1. Hypertext Markup Language (HTML).....3
2.1.2. Elemen dan Tags.....3
2.2. Proses Parsing HTML.....4
2.3. DOM Tree.....5
2.4. CSS Selector.....5
2.4.1. Kategori Selector Dasar.....5
3. BAB 3 Aplikasi BFS dan DFS.....7
3.1. BFS.....7
3.2. DFS.....8
3.4. Selector.....10
3.4.1. Struktur Data.....10
3.4.2. Tokenisasi.....12
3.4.3. Parsing.....13
3.4.4. Matching.....15
4. BAB 4 Implementasi dan Pengujian.....17
5. BAB 5 Kesimpulan.....35

1. BAB 1 Deskripsi

1.1. Latar Belakang

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk *tree* yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

Struktur sebuah file HTML pada umumnya meliputi elemen root <html> yang berisi dua bagian utama yaitu <head> dan <body>. Bagian <head> berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan <body> berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

1.2. Tugas

Membuat program untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

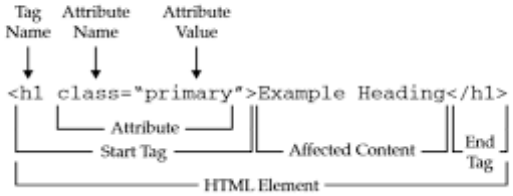
2. BAB 2 Landasan Teori

2.1. Struktur Dasar Web

2.1.1. Hypertext Markup Language (HTML)

Hypertext Markup Language (HTML) adalah bahasa markup yaitu sistem pengkodean berbasis teks untuk menspesifikasikan struktur dan format dari sebuah dokumen dengan hubungan antar bagiannya yang dipakai untuk menampilkan konten dan struktur pada web. HTML ini biasanya digunakan bersamaan dengan Cascading Style Sheets (CSS) dan bahasa pemrograman web seperti Java scripts.

2.1.2. Elemen dan Tags



https://tutorial.techaltum.com/htmlTags.html

Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web.

Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (< >). Berdasarkan struktur penulisannya, elemen HTMl secara umum dapat diklasifikasikan dalam dua kategori

1. Elemen Berpasangan
   Sebagian besar elemen HTML membutuhkan tag pembuka dan tag penutup untuk membatasi cakupan konten. Tag pembuka menandai titik awal suatu elemen, sedangkan tag penutup menandai titik akhirnya. Tag penutup memiliki sintaksis yang identik dengan tag pembuka, namun diwajibkan untuk menambahkan karakter garis miring (/) tepat sebelum nama tag (sebagai contoh: untuk membuka paragraf dan untuk menutupnya).
2. Elemen Kosong (*void elements/self-closing tags*)

Terdapat beberapa elemen HTML yang beroperasi secara mandiri dan tidak memiliki tag penutup. Elemen ini disebut elemen kosong karena tidak didesain untuk membungkus konten teks atau elemen turunan lainnya (sebagai contoh: <img> untuk menyematkan gambar, atau <br> untuk memberikan jeda baris).

2.2. Proses Parsing HTML

Untuk tokenisasi, kami menggunakan library bawaan dari golang untuk tokenisasi masing-masing tag html sehingga kami tidak melakukan string matching manual untuk tiap tags.

Parsing dilakukan dengan data struktur *LIFO* dari *stack* (dapat dilihat dari source code bahwa kami tidak menggunakan stack bawaan dari Golang, melainkan hanya menggunakan array biasa yang menerapkan *LIFO)*. Secara singkat, pembuatan tree dilakukan dengan memasukkan sebuah tag opening dalam bentuk *node* pada *stack.* Semua *tag* yang masuk setelahnya pada *stack* akan menjadi anak dari *node* yang sebelumnya sudah masuk. Contoh, apabila stack berisi = {1, 2, 3, 4} maka 4 adalah anak dari 3, 3 anak dari 2, dan seterusnya. Apabila stack berisi {1, 2}, dan kemudian ditemukan tag penutup dari node 2, maka 2 akan di-*pop* dan tersisa {1} saja. Apabila terdapat *tag* baru, maka *stack* menjadi {1,3}, dan melalui proses ini, anak dari *node* 1 menjadi *node 2* dan *node* 3.

Terdapat beberapa *exception* yang harus diperhatikan. Contohnya dalam HTML terdapat *tags* yang *self-closing*, artinya *tag* tersebut tidak menunggu sebuah *closing-tag*. Dalam *parser* maka tag seperti ini tidak dimasukkan *stack* sama sekali.

Exception handling juga menangani apabila HTML yang diberikan tidak memiliki format yang benar. Contohnya yaitu *tags* yang tidak ditutup atau *stray tags* di awal (pada setiap dokumen HTML diekspektasikan awalan *tag* HTML itu sendiri). Untuk *handling stray tags* cukup mudah dengan mengabaikan semua *tag* sebelum ditemukan *tag* HTML. Untuk mengatasi *tags* yang tidak ditutup, perlu dilakukan beberapa algoritma sederhana pada *stack*. Contoh terdapat "<a> <b> <c> </a>", maka *tag* <b> dan <c> ditengah akan otomatis di-*pop* juga dari stack. Algoritma yang dilakukan adalah untuk suatu *tag* penutup yang ditemukan, akan dicari traversal ke belakang dari atas *stack* untuk menemukan *tag* pembuka yang cocok pertama. Segala tag yang ditengah antara *tag* pembuka dan penutup akan di-*pop* juga.

2.3. DOM Tree

Document Object Model (DOM) adalah representasi struktur dari dokumen HTML dalam bentuk tree atau pohon. Setiap elemen HTML akan merupakan sebuah node dengan relasi antar node adalah hubungan parent-child pada HTML. Setiap node dapat menyimpan informasi yang dimiliki sebuah elemen HTML seperti tag, atribu, class, parent, dan children. Struktur ini akan selanjutnya diproses oleh algoritma traversal.

2.4. CSS Selector

CSS Selector merupakan mekanisme ekspresif yang digunakan untuk mengidentifikasi dan menyeleksi elemen spesifik di dalam struktur DOM (Document Object Model). Secara fundamental, selector berfungsi untuk menentukan elemen target yang akan dimanipulasi melalui deklarasi properti visual. Dalam konteks manipulasi DOM dan traversal menggunakan JavaScript, mesin selector (seperti metode querySelector atau querySelectorAll) memfasilitasi pencarian simpul (node) secara presisi berdasarkan atribut, relasi, atau posisinya di dalam hierarki dokumen.

2.4.1. Kategori Selector Dasar

Beberapa jenis *selector* mendasar yang umum digunakan adalah

1. Tag *Selector*

   Menyeleksi elemen berdasarkan nama tag HTML secara spesifik.

2. *Class Selector*

   Menyeleksi elemen berdasarkan nilai atribut class yang dimilikinya. Selektor ini dideklarasikan dengan menggunakan prefiks tanda titik (.).

3. *ID Selector*

   Menyeleksi sebuah elemen unik berdasarkan nilai atribut id. Karena atribut ID bersifat tunggal di dalam satu dokumen, selektor ini memiliki tingkat spesifisitas yang tinggi. Deklarasinya menggunakan prefiks tanda pagar (#).

4. *Universal Selector*

   Menyeleksi keseluruhan elemen tanpa pengecualian di dalam struktur DOM. Selektor ini direpresentasikan dengan simbol asterisk (*).

2.4.2. Kombinator

Kombinator adalah operator sintaksis yang digunakan untuk menggabungkan beberapa selektor dasar guna mendefinisikan hubungan struktural yang lebih kompleks di antara elemen-elemen tersebut. Konsep kombinator ini secara

langsung merepresentasikan hubungan hirarkis antar-simpul pada DOM *Tree*. Jenis-jenis kombinator meliputi:

1. Descendant Combinator
   Direpresentasikan dengan karakter spasi ( ). Berfungsi menyeleksi seluruh elemen turunan dari sebuah elemen referensi, terlepas dari tingkat kedalamannya di dalam hierarki bersarang.
2. Child Combinator
   Direpresentasikan dengan karakter lebih besar dari (>). Berfungsi menyeleksi elemen turunan yang posisinya persis satu tingkat di bawah elemen induk (direct child).
3. Adjacent Sibling Combinator
   Direpresentasikan dengan karakter tambah (+). Berfungsi menyeleksi sebuah elemen yang posisinya tepat bersebelahan setelah elemen referensi, dengan syarat keduanya berada pada tingkat hierarki (induk) yang sama.
4. General Sibling Combinator
   Direpresentasikan dengan karakter tilde (~). Berfungsi menyeleksi seluruh elemen yang berada pada tingkat hierarki yang sama dan berposisi setelah elemen referensi, meskipun tidak bersebelahan secara langsung.

2.5. Algoritma Traversal Traversal pada DOM

Dalam proses penelusuran (*traversal*) struktur data graf atau pohon (*tree*), termasuk pada hierarki *Document Object Model* (DOM), terdapat dua pendekatan algoritma fundamental yang digunakan, yaitu *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS). Kedua algoritma ini beroperasi secara sistematis untuk mengunjungi setiap simpul (*node*), namun memiliki perbedaan signifikan pada mekanisme struktural dan implementasi memori.

2.5.1. *Breadth-First Search* (BFS)

Breadth-First Search (Pencarian Melebar) adalah algoritma traversal yang menelusuri struktur pohon secara horizontal atau tingkat demi tingkat (*level by level*). Algoritma ini akan mengunjungi seluruh simpul pada tingkat kedalaman saat ini sebelum melanjutkan penelusuran ke tingkat kedalaman berikutnya.

BFS ini menggunakan struktur data *queue* yang menggunakan prinsip *First In First Out* (FIFO), di mana simpul yang pertama kali dimasukkan ke dalam antrean akan menjadi simpul pertama yang diproses.

2.5.2. *Depth-First Search* (DFS)

Depth-First Search (Pencarian Mendalam) adalah algoritma traversal yang memulai penelusuran dari simpul awal dan berjalan menyusuri salah satu cabang sejauh mungkin secara vertikal hingga mencapai simpul daun (*leaf node*), yakni

simpul yang tidak lagi memiliki tetangga yang belum dikunjungi. Setelah mencapai ujung, algoritma akan melakukan proses mundur (*backtracking*) untuk menelusuri cabang lain yang belum terjamah.

DFS menggunakan struktur data *stack* baik secara iterasi atau dengan rekursif, yang menggunakan prinsip *Last In First Out* (LIFO), di mana simpul terakhir yang ditemui adalah simpul pertama yang diproses lebih lanjut. Pendekatan ini secara konseptual membangun penelusuran berdasarkan sub-pohon demi sub-pohon (*sub-tree by sub-tree*).

3. BAB 3 Aplikasi BFS dan DFS

3.1. BFS

BFS pada sistem ini diimplementasikan pada traversal DOM tree untuk mencari node yang sesuai dengan selector yang sudah diparsing menjadi suatu data type yang bisa dibandingkan dengan node DOM Tree. BFS sendiri diimplementasikan pada Fungsi RunBFS yang menerima data bertipe DOM Tree, string selector dan juga integer untuk menyatakan limit (batas jumlah hasil, ada karena memperhitungan HTML dengan size besar) sebagai berikut:

```
func RunBFS(tree *models.DOMTree, selector string, limit int) Res
```

lalu fungsi ini akan mengembalikan struct Result bertipe sebagai berikut:

```
type Res struct {
    ID          []int `json:"traversalOrder"`
    VisitedOrder []int  `json:"visitedOrder"`
    VisitLength int   `json:"traversalLength"`
    Algorithm   string `json:"algorithm"`
}
```

Sebelum traversal dimulai string selector diparse menjadi struktur MultiSelector menggunakan gungsi ParseMultiSelector dan juga menginisialisasi queue dengan root node sebagai objek traversal pertamanya.

```
        nodeID := queue[0]
         queue = queue[1:]

         node := tree.Nodes[nodeID]
```

```
        if node == nil {
            continue
        }

        result.VisitedOrder = append(result.VisitedOrder, nodeID)
        result.VisitLength++

        if MatchesAnySelector(tree, nodeID, multi) {
            result.ID = append(result.ID, nodeID)
        }

        if limit > 0 && len(result.ID) >= limit {
            break
        }

        for _, childID := range node.Children {
            queue = append(queue, childID)
        }
```

Pada setiap iterasi, node terdepan di-*dequeue*. Node tersebut dicatat ke VisitedOrder dan penghitung VisitLength ditambah satu, tanpa memandang apakah node cocok dengan selector atau tidak. Setelah itu, MatchesAnySelector() dipanggil untuk memeriksa kecocokan. Jika cocok, node dimasukkan ke result.ID. Seluruh anak dari node saat ini kemudian dienqueue ke belakang queue. Queue memastikan bahwa semua node pada suatu level X akan diproses duluan sebelum node pada levek X+1. Setelah seluruh node ter-*exhausted* dan queue BFS kosong. BFS menghasilkan urutan kunjungan yang melebar secara horizontal, mengikuti struktur DOM Tree

3.2. DFS

Level implementasi DFS pada sistem ini sama seperti level implementasi BFS, DFS melakukan traversal ke semua node dengan menggunakan fungsi MatchesAnySelector sebagai komparator Node valid, perbedaan utama adalah sistem penyimpanan node traversal dalam sebuah stack, bukan queue.

```
func RunDFS(tree *models.DOMTree, selector string, limit int) Res
```

Sebelum traversal, diinisialisasi MultiSelector dan juga Stack.

```
        n := len(stack) - 1
        nodeID := stack[n]
```

```
        stack = stack[:n]

        node := tree.Nodes[nodeID]
        if node == nil {
            continue
        }

        result.VisitedOrder = append(result.VisitedOrder, nodeID)
        result.VisitLength++

        if MatchesAnySelector(tree, nodeID, multi) {
            result.ID = append(result.ID, nodeID)
        }

        if limit > 0 && len(result.ID) >= limit {
            break
        }

        for i := len(node.Children) - 1; i >= 0; i-- {
            stack = append(stack, node.Children[i])
        }
```

Pada setiap iterasi, node teratas diambil dari atas stack. Seperti pada BFS, node tersebut langsung dicatat ke VisitedOrder dan VisitLength tanpa bergantung pada hasil pencocokan selector. Pencocokan terhadap selector dilakukan setelahnya. Proses penambahan node ke stack dilakukan dengan urutan terbalik (dari node dengan id terbelekang) karena penggunaan stack, maka selalu diinginkan node yang traversal paling awal untuk di pop terlebih dahulu.

3.3. Analisis Kompleksitas Algoritma BFS dan DFS

Dalam mengevaluasi dan membandingkan kinerja algoritma penelusuran (*traversal*), dua parameter analitik utama yang digunakan adalah kompleksitas waktu (*time complexity*) dan kompleksitas ruang (*space complexity*).

3.3.1. Kompleksitas waktu

Secara teoretis, baik *Breadth-First Search* (BFS) maupun *Depth-First Search* (DFS) memiliki tingkat efisiensi waktu linear yang identik ketika diterapkan pada struktur data grafik atau pohon. Kompleksitasnya O(V + E) dengan Variabel V merepresentasikan total jumlah simpul, dan E merepresentasikan total jumlah sisi.

3.3.2. Kompleksitas Ruang (Memori)

1. Kompleksitas Memori BFS
   Kompleksitas memori BFS berbanding lurus dengan tingkat kelebaran maksimum (*maximum width*) dari struktur pohon. Hal ini disebabkan antrean (*queue*) harus menyimpan seluruh referensi simpul pada satu tingkat kedalaman yang sama secara bersamaan sebelum berpindah ke tingkat berikutnya.
   Kompleksitas ruangnya dideklarasikan sebagai O(W), di mana W adalah jumlah elemen maksimum pada satu level (contoh kasus terburuk: elemen <ul> yang memuat ribuan anak <li> sejajar).

2. Analisis Ruang DFS
   Kompleksitas memori DFS berbanding lurus dengan tingkat kedalaman maksimum (maximum depth/height) dari struktur pohon. Memori digunakan untuk menjaga status tumpukan eksekusi (call stack) dari simpul akar hingga mencapai simpul daun terdalam. Kompleksitas ruangnya dideklarasikan sebagai O(H), di mana H adalah tingkat kedalaman pohon (contoh kasus terburuk: puluhan tag yang disusun bersarang/ nested ke dalam).

3.4. Selector

Sistem selektor dibagi menjadi 3 tahap utama: tokenisasi string selektor, parsing menjadi struktur data selector, dan matching dengan node. Selector memegang fungsi node matching utama yang dipakai oleh traversal DFS dan BFS yaitu fungsi MatchesAnySelector().

3.4.1. Struktur Data

Selector direpresentasikan menjadi struktur data spesifik. Satu selector sederhana (base) akan disimpan dalam struct SelectorType

```
type SelectorType struct {
    Tag        string
    Classes    []string
    ID         string
    Attributes []AttributeSelector
    All        bool // Specifically for *
}
```

Untuk struktur atribut maka digunakan struct AttributeSelector

```
type AttributeSelector struct {
    Name     string
    Operator string
    Value    string
}
```

Jika selector mengandung combinator maka satu query disimpan dalam struct SelectorPair yang menyatakan hubungan antara 2 selektor, lalu disimpan dalam struct SelectorChain.

```
// selector step
// this record type and then their relation symbol
// like div + p -> {}'div', '+'}, {'p', ""}
type SelectorPair struct {
    Sel      SelectorType
    Relation SelectorRelation
}

// stores a bunch of selector pair and chain them when processing
in matching
type SelectorChain struct {
    Steps []SelectorPair
}

// for ors (a,b,c)
type MultiSelector struct {
    Chains []SelectorChain
}

type SelectorRelation string

const (
    NoRelation      SelectorRelation = ""
    Descendant      SelectorRelation = " "
    Child           SelectorRelation = ">"
    AdjacentSibling SelectorRelation = "+"
    GeneralSibling  SelectorRelation = "~"
)
```

Dan tentunya ada struct multiSelector untuk menyimpan selector yang multiQuery, dipisahkan oleh koma.

Contoh Representasi

div > p
akan disimpan menjadi
SelectorPair:
[
  {Sel (SelectorType): div, Relation: “>”},
  {Sel (SelectorType): p, Relation: “” //empty}
]

3.4.2. Tokenisasi

Tahap pertama dalam selector adalah tokenisasi, yaitu memecah string selector menjadi token-token yang lebih kecil. Fungsi Tokenize digunakan untuk memecah satu selector tanpa koma berdasarkan separator seperti spasi, >, +, ~, dan blok atribut [...].

```
func Tokenize(s string) []string {
    s = strings.TrimSpace(s)
    if len(s) == 0 {
        return nil
    }

    tokens := []string{}
    descendant := false // detect whether white space is for child
operator div p (and not skipped)

    for i := 0; i < len(s); {
        c := s[i]
        if c == ' ' || c == '\t' {
            descendant = true
            i++
            continue
        }

        if c == '>' || c == '+' || c == '~' {
            tokens = append(tokens, string(c))
            descendant = false
            i++
            continue
        } ...
```

Pada proses ini, terdapat variabel descendant yang digunakan untuk membedakan spasi biasa dengan spasi yang digunakan sebagai combinator descendant pada contoh query ‘div p’ yang berarti semua node p yang memiliki ancestor div.

```
        j := i
        depth := 0
        for j < len(s) {
            if s[j] == '[' {
                depth++
            } else if s[j] == ']' && depth > 0 {
                depth--
            }

            if depth == 0 && (s[j] == ' ' || s[j] == '\t' || s[j]
== '>' || s[j] == '+' || s[j] == '~') {
                break
            }
            j++
        }
```

Fungsi ini juga menyimpan depth pembacaan atribut agar karakter di dalam atribut tidak salah dianggap sebagai pemisah selector. Untuk multiselector yang dipisahkan koma maka digunakan fungsiTokenizeByComma(), dimana terdapat depth tracking pada fungsi ini mencegah koma di dalam atribut dianggap sebagai pemisah multi-selector.

3.4.3. Parsing

Parsing selector dilakukan untuk konversi token menjadi struktur data selector yang benar dan digunakan untuk proses matching.

```
func ParseSelector(selector string) []SelectorPair {
    tokens := Tokenize(selector)
    if len(tokens) == 0 {
        return nil
    }

    type partialStep struct {
        sel SelectorType
        rel SelectorRelation
    }
```

```go
    steps := make([]partialStep, 0)
    pendingRel := Descendant

    for _, token := range tokens {
        if token == "" {
            continue
        }

        switch token {
        case " ":
            pendingRel = Descendant
            continue
        case ">":
            pendingRel = Child
            continue
        case "+":
            pendingRel = AdjacentSibling
            continue
        case "~":
            pendingRel = GeneralSibling
            continue
        }

        if len(steps) > 0 {
            steps[len(steps)-1].rel = pendingRel
        }

        steps = append(steps, partialStep{
            sel: DetectSelectorType(token),
            rel: NoRelation,
        })

        pendingRel = Descendant
    }

    result := make([]SelectorPair, len(steps))
    for i, step := range steps {
        result[i] = SelectorPair{Sel: step.sel, Relation: step.rel}
    }

    return result
}
```

Variabel pendingRel digunakan untuk menyimpan relasi sementara yang ditemukan sebelum selector berikutnya diproses. Ketika parser menemukan token combinator seperti >, +, atau ~, parser belum langsung dapat memasukkannya ke selector berikutnya. Relasi tersebut harus dipasang pada selector sebelumnya, karena relasi menggambarkan

hubungan dari selector sebelumnya ke selector setelahnya. Selain itu terdapat default relation yang di set saat

```
    steps := make([]partialStep, 0)
    pendingRel := Descendant
```

Jika dua selector sederhana dipisahkan oleh spasi, maka relasinya adalah Descendant. Deteksi Tipe selektor dilakuakn oleh DetectSelectorType() yang dipakai untuk mengisi struct SelectorType().

3.4.4. Matching

Node matching dilakukan dengan fungsi MatchNode()

```
func MatchNode(node *models.Node, selector SelectorType) bool {
    if node == nil {
        return false
    }
    if selector.All && selector.Tag == "" && selector.ID == "" &&
len(selector.Classes) == 0 && len(selector.Attributes) == 0 {
        return true
    }

    if selector.Tag != "" && selector.Tag != node.Tag {
        return false
    }
    if selector.ID != "" {
        if node.Attributes == nil || node.Attributes["id"] !=
selector.ID {
            return false
        }
    }

    for _, attr := range selector.Attributes {
        if !MatchAttribute(node, attr) {
            return false
        }
    }

    for _, want := range selector.Classes {
        found := false
        for _, have := range node.Classes {
            if have == want {
                found = true
                break
            }
```

```
        }
        if !found {
            return false
        }
    }

    return true
}
```

Pada sistem ini, node matching tidak menentukan urutan traversal. Urutan traversal tetap ditentukan oleh BFS atau DFS. Node matching hanya mengevaluasi node yang sedang dikunjungi dan mengembalikan nilai boolean true jika cocok dan false jika tidak cocok.

Ururtan pencocokan suatu node adalah:

- Tag
- ID
- Class

```
func MatchAttribute(node *models.Node, attr AttributeSelector)
bool {
    if node == nil || node.Attributes == nil {
        return false
    }

    val, exists := node.Attributes[attr.Name]
    if !exists {
        return false
    }

    switch attr.Operator {
    case "":
        return true
    case "=":
        return val == attr.Value
    case "^=":
        return strings.HasPrefix(val, attr.Value)
    case "$=":
        return strings.HasSuffix(val, attr.Value)
    case "*=":
        return strings.Contains(val, attr.Value)
    case "~=":
        for _, word := range strings.Fields(val) {
            if word == attr.Value {
                return true
            }
        }
```

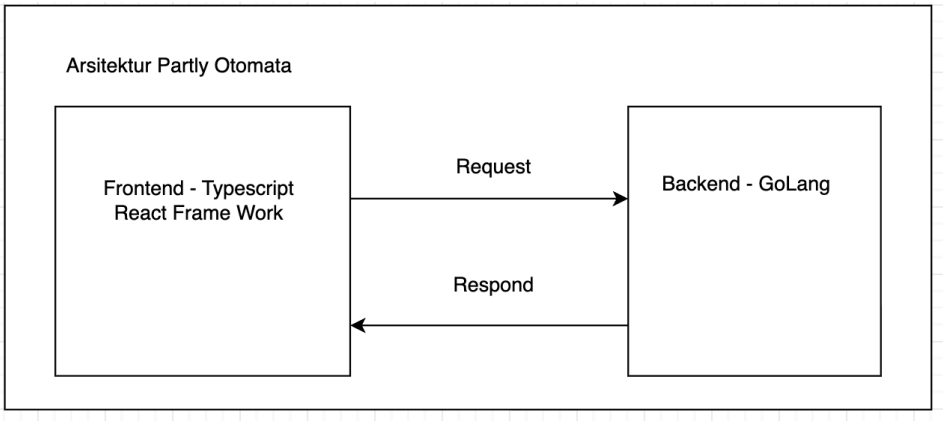
```
        return false
    case "|=":
        return val == attr.Value || strings.HasPrefix(val,
attr.Value+"-")
    default:
        return false
    }
}
```

- Atribut (dengan fungsi MatchAttribute)

JIka dalam proses matching, satu kondisi node tidak sesuai dengan struct selector, maka akan langsung ditolak.

4. BAB 4 Implementasi dan Pengujian

4.1. Implementasi



Secara gambaran umum program kami dapat dibagi menjadi dua bagian

1. Antarmuka Pengguna (*Front-end*)
   Subsistem ini berfungsi sebagai lapisan presentasi (*presentation layer*) yang beroperasi pada sisi klien. Tanggung jawab utama dari *front-end* adalah menerima interaksi pengguna dan merender representasi visual secara dinamis, khususnya dalam menampilkan bentuk grafis dari hasil *parsing* struktur DOM *Tree* serta memvisualisasikan hasil penelusuran algoritma.
2. Sisi Server (*Back-end*)
   Subsistem ini bertindak sebagai mesin komputasi utama (*processing layer*). *Back-end* bertanggung jawab penuh atas pelaksanaan logika inti program, yang mencakup operasi komputasional, kalkulasi metrik, serta

eksekusi algoritma traversal (seperti pencarian pada simpul-simpul DOM) sebelum hasil akhirnya dikembalikan ke sisi klien untuk divisualisasikan.

4.2. Pengujian

Struktur HTML internal untuk melakukan testing selector

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Testing</title>
</head>
<body>
  <h1>Testing</h1>

  <div class="intro">
    <p>one <span id="Lastname">two</span></p>
    <p id="my-Address">three</p>
    <section>
      <p>four</p>
    </section>
  </div>

  <ul id="friend-list">
    <li>a</li>
    <li>b</li>
    <li>c</li>
  </ul>

  <p>five</p>

  <table id="stats-table" border="1">
    <tr>
      <th>x</th>
      <th>y</th>
    </tr>
    <tr>
      <td>1</td>
      <td>2</td>
    </tr>
  </table>

  <p class="myquote" title="hello beautiful world">six</p>
</body>
```

```
</html>
```

Testing

one two

three

four

- a
- b
- c

five

x	y
1	2

six

Selector yang akan diuji

```
.intro
```

Testing

one two

three

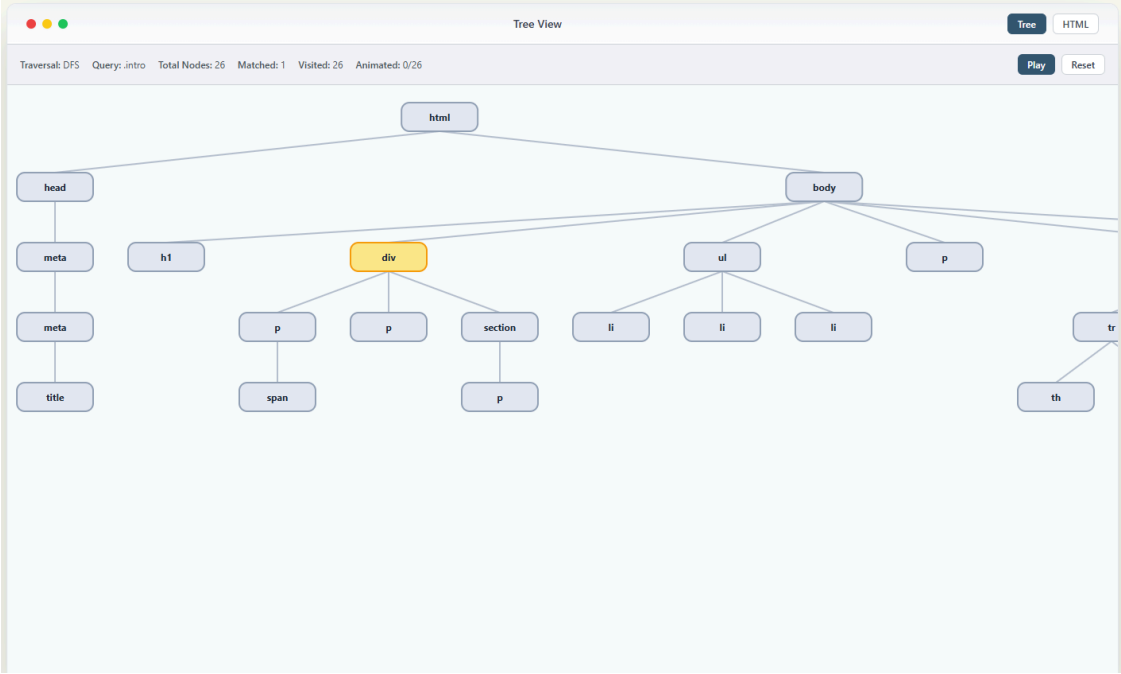
four

- a
- b
- c

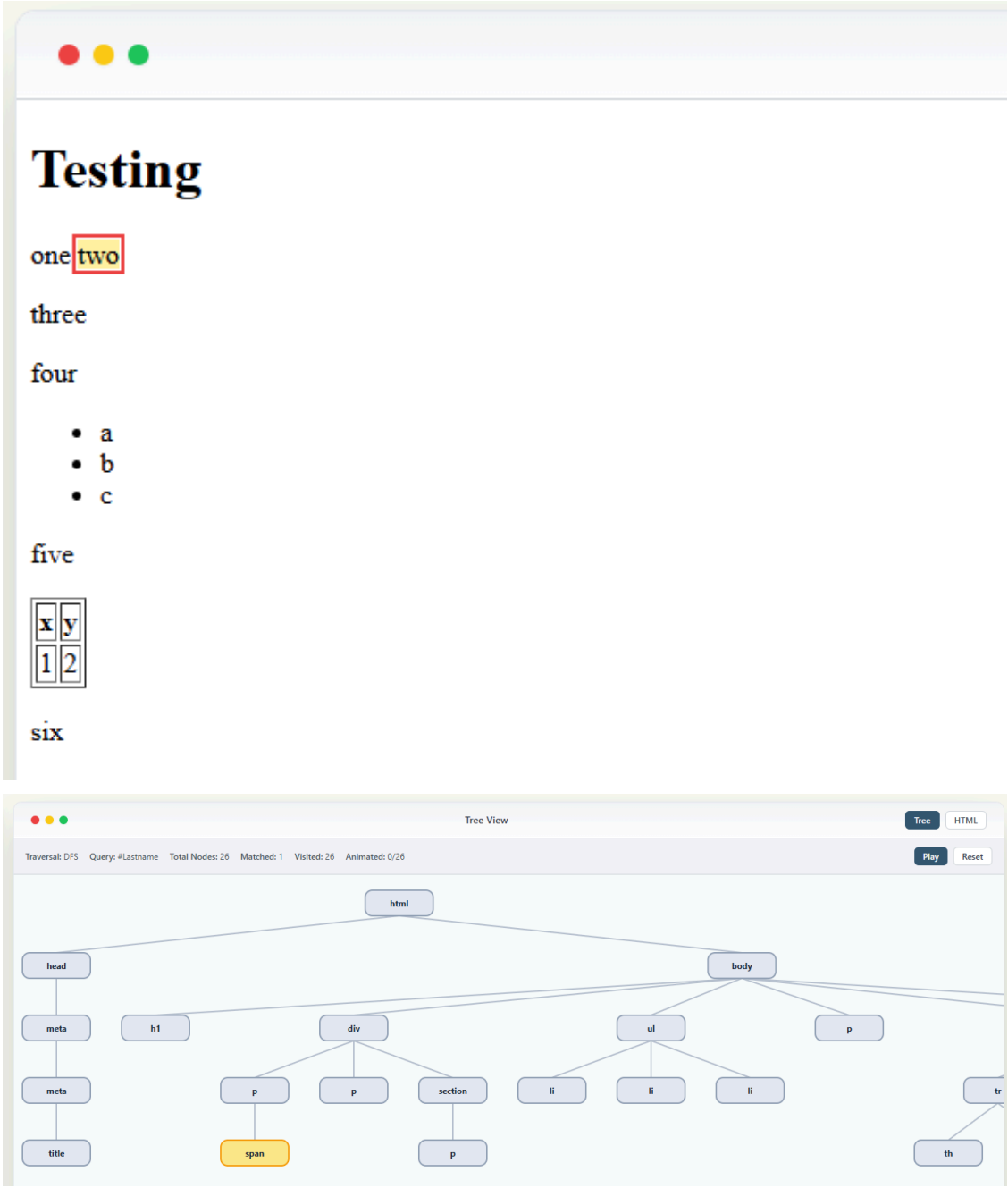
five

x	y
1	2

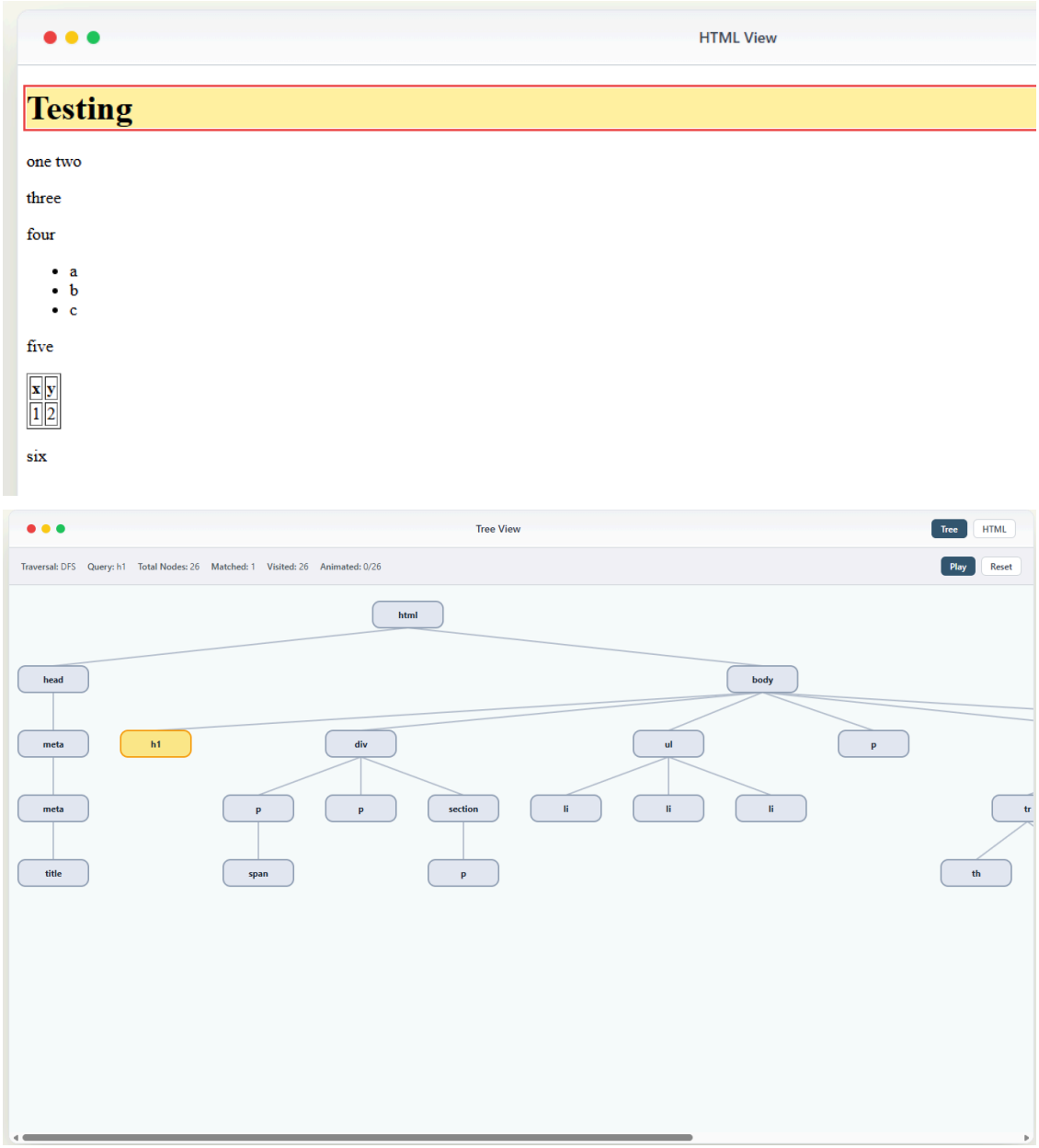
six



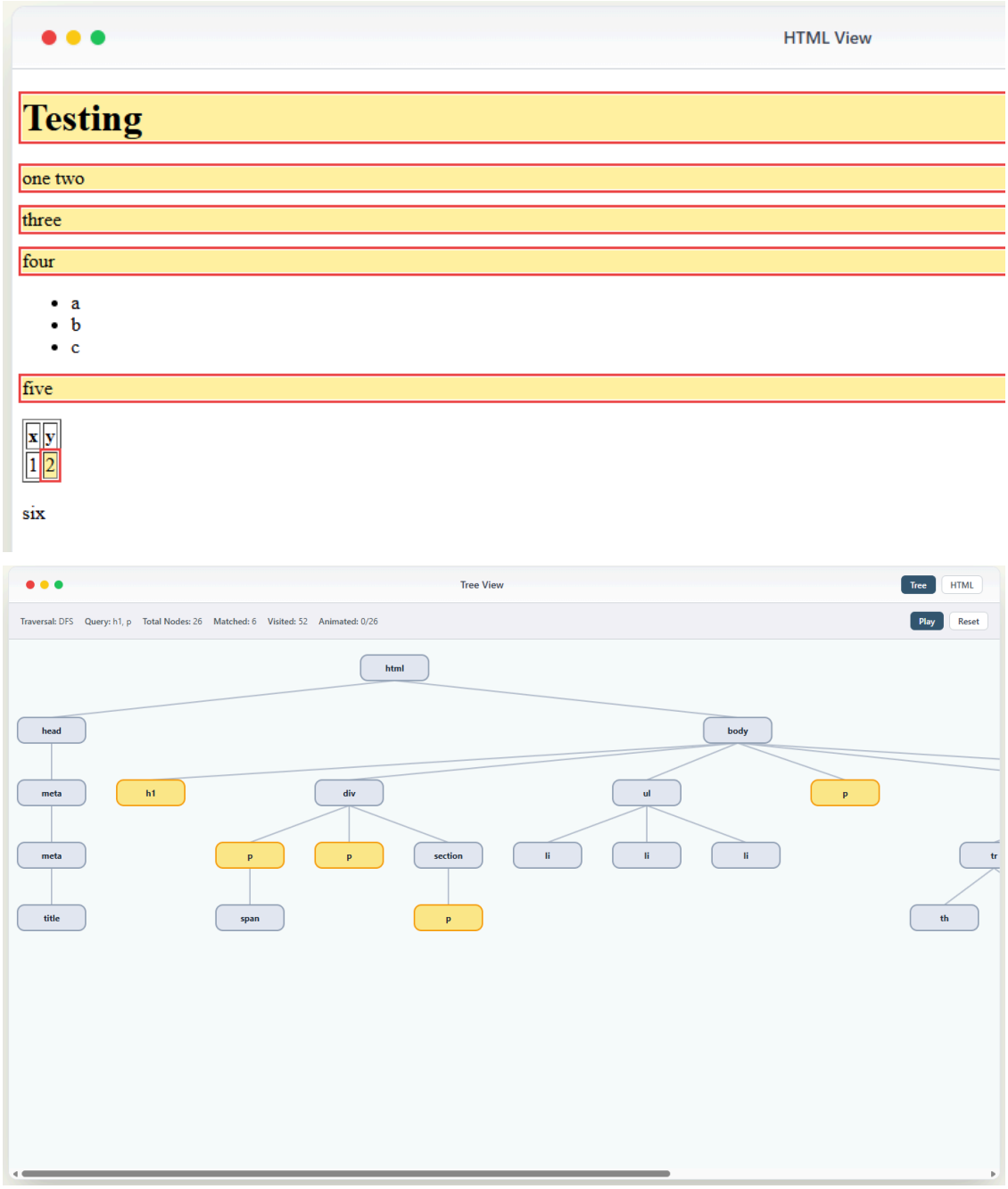
#Lastname


Testing
one two
three
four
a
b
c
five
x y
1 2
six
Tree View
Tree
HTML
Traversal: DFS
Query: #Lastname
Total Nodes: 26
Matched: 1
Visited: 26
Animated: 0/26
Play
Reset
html
head
body
meta
h1
div
ul
p
meta
p
p
section
li
li
li
tr
title
span
p
th

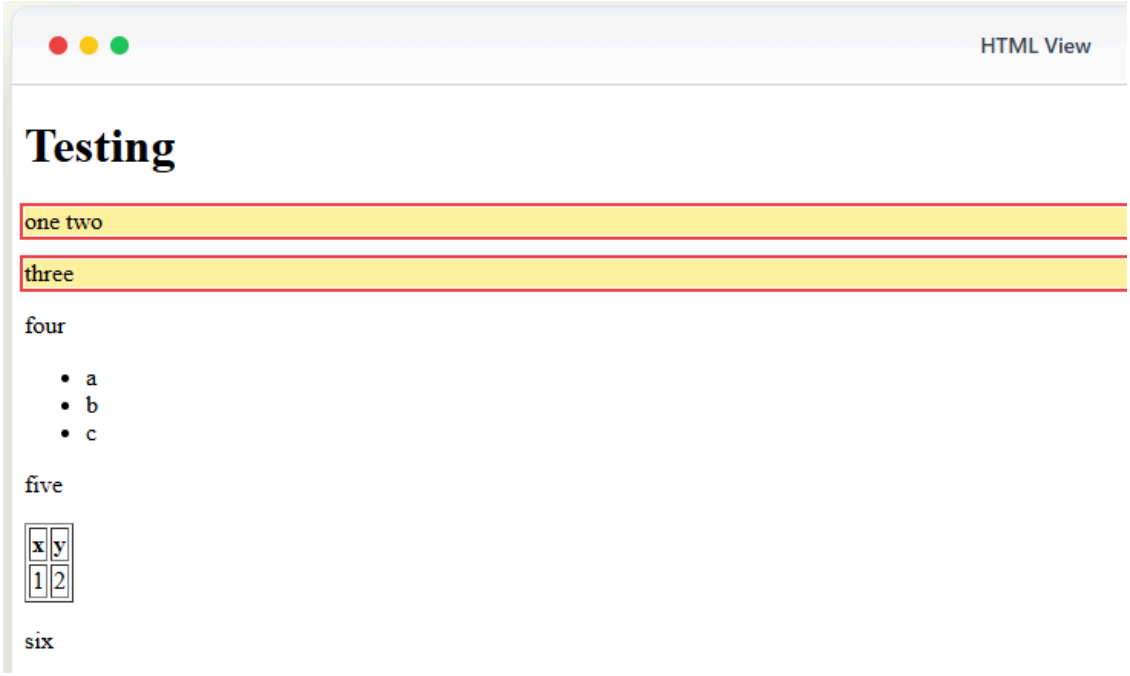
h1

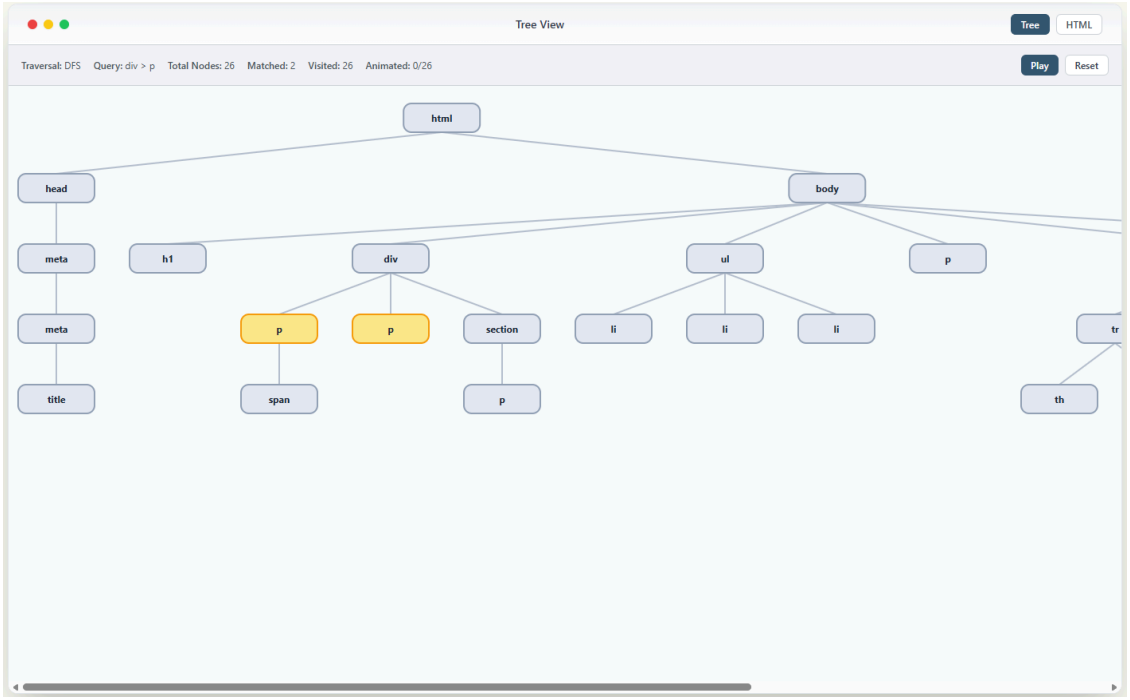

HTML View
Testing
one two
three
four
a
b
c
five
x y
1 2
six
Tree View
Tree
HTML
Traversal: DFS
Query: h1
Total Nodes: 26
Matched: 1
Visited: 26
Animated: 0/26
Play
Reset
html
head
body
meta
h1
div
ul
p
meta
p
p
section
li
li
li
tr
title
span
p
th

h1, p

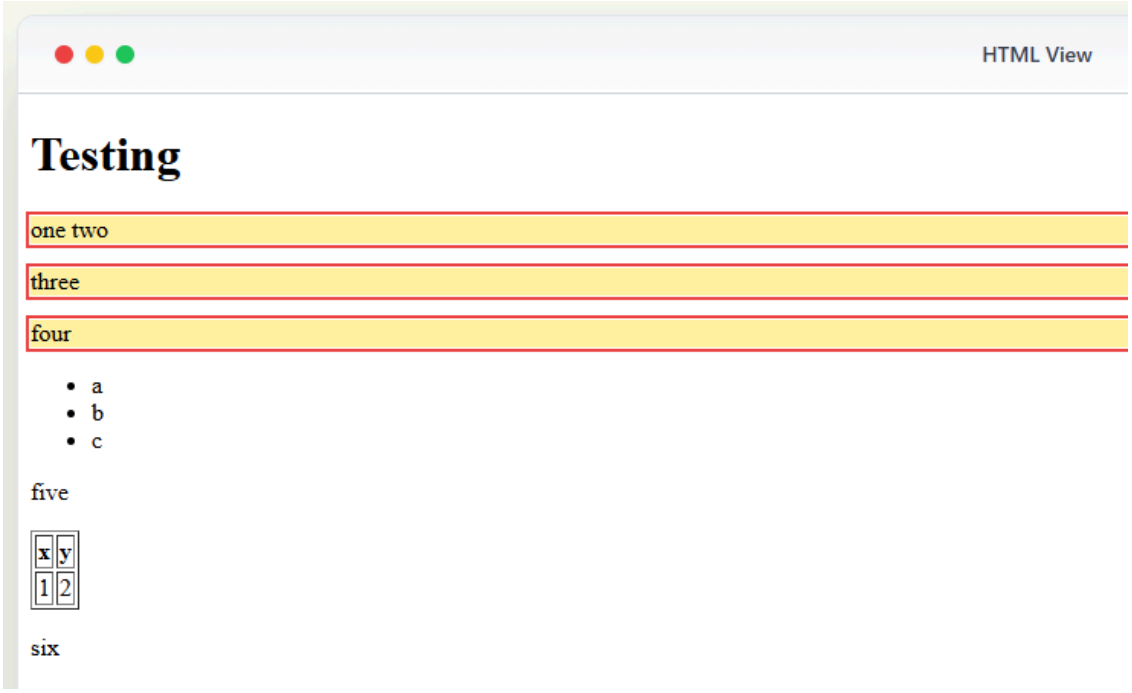

HTML View
Testing
one two
three
four
a
b
c
five
x
y
1
2
six
Tree View
Tree
HTML
Traversal: DFS
Query: h1, p
Total Nodes: 26
Matched: 6
Visited: 52
Animated: 0/26
Play
Reset
html
head
body
meta
h1
div
ul
p
meta
p
p
section
li
li
li
tr
title
span
p
th

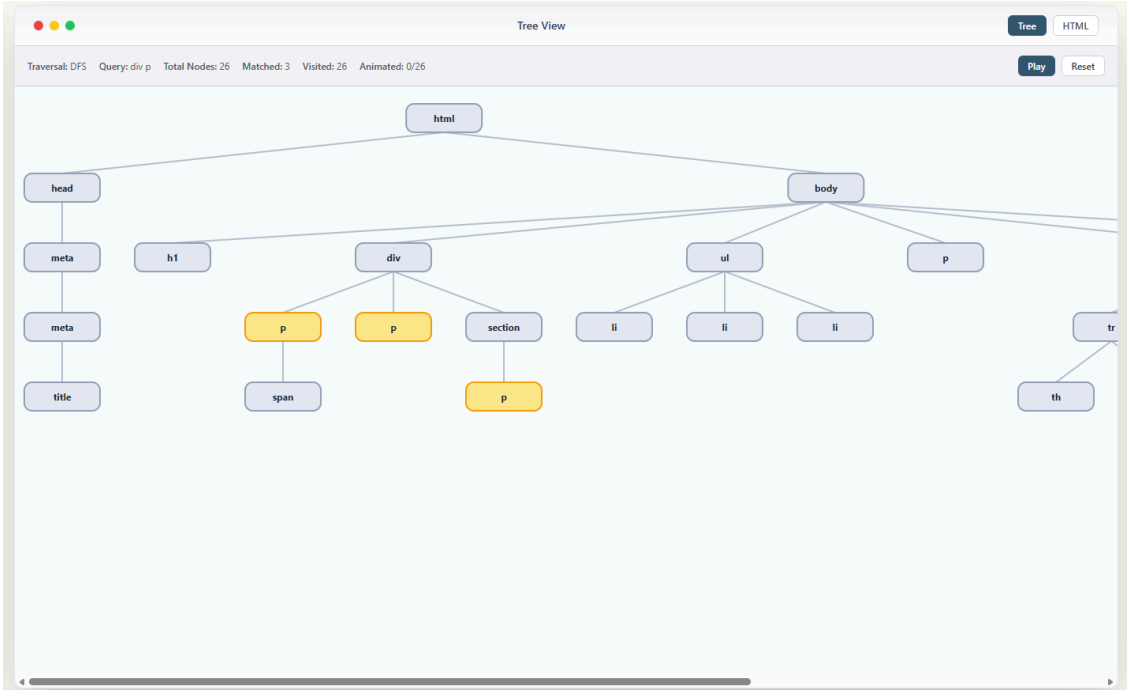
div > p


HTML View
Testing
one two
three
four
a
b
c
five
x y
1 2
six

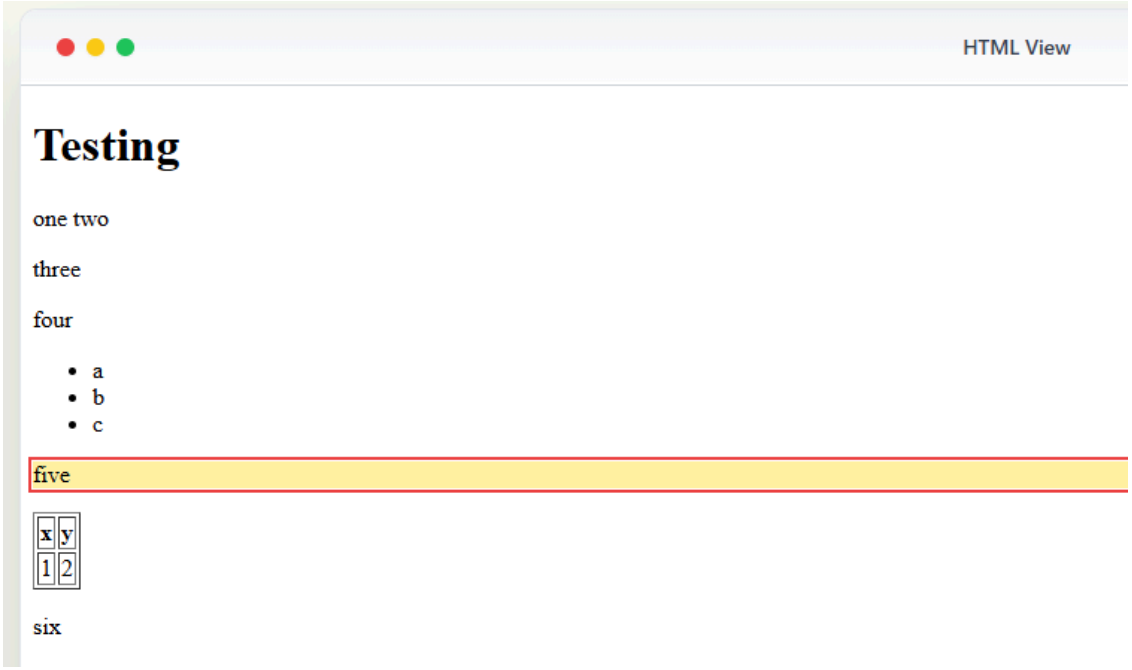

Tree View
Tree
HTML
Traversal: DFS
Query: div > p
Total Nodes: 26
Matched: 2
Visited: 26
Animated: 0/26
Play
Reset
html
head
body
meta
h1
div
ul
p
meta
p
p
section
li
li
li
tr
title
span
p
th

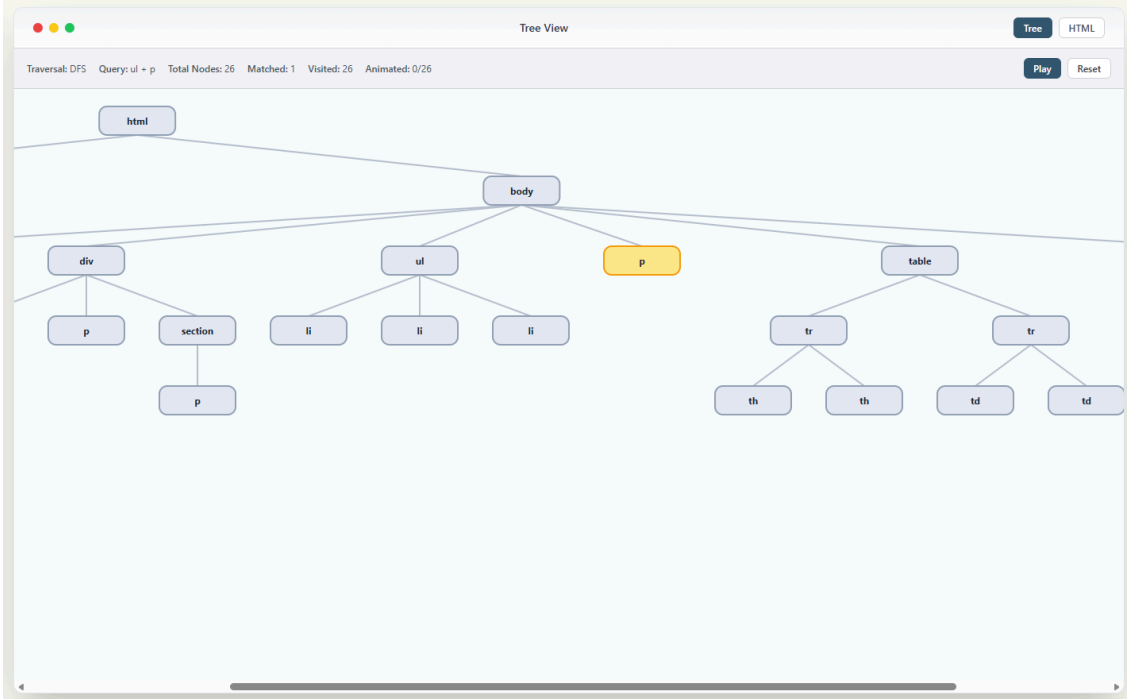
div p


HTML View
Testing
one two
three
four
a
b
c
five
x y
1 2
six

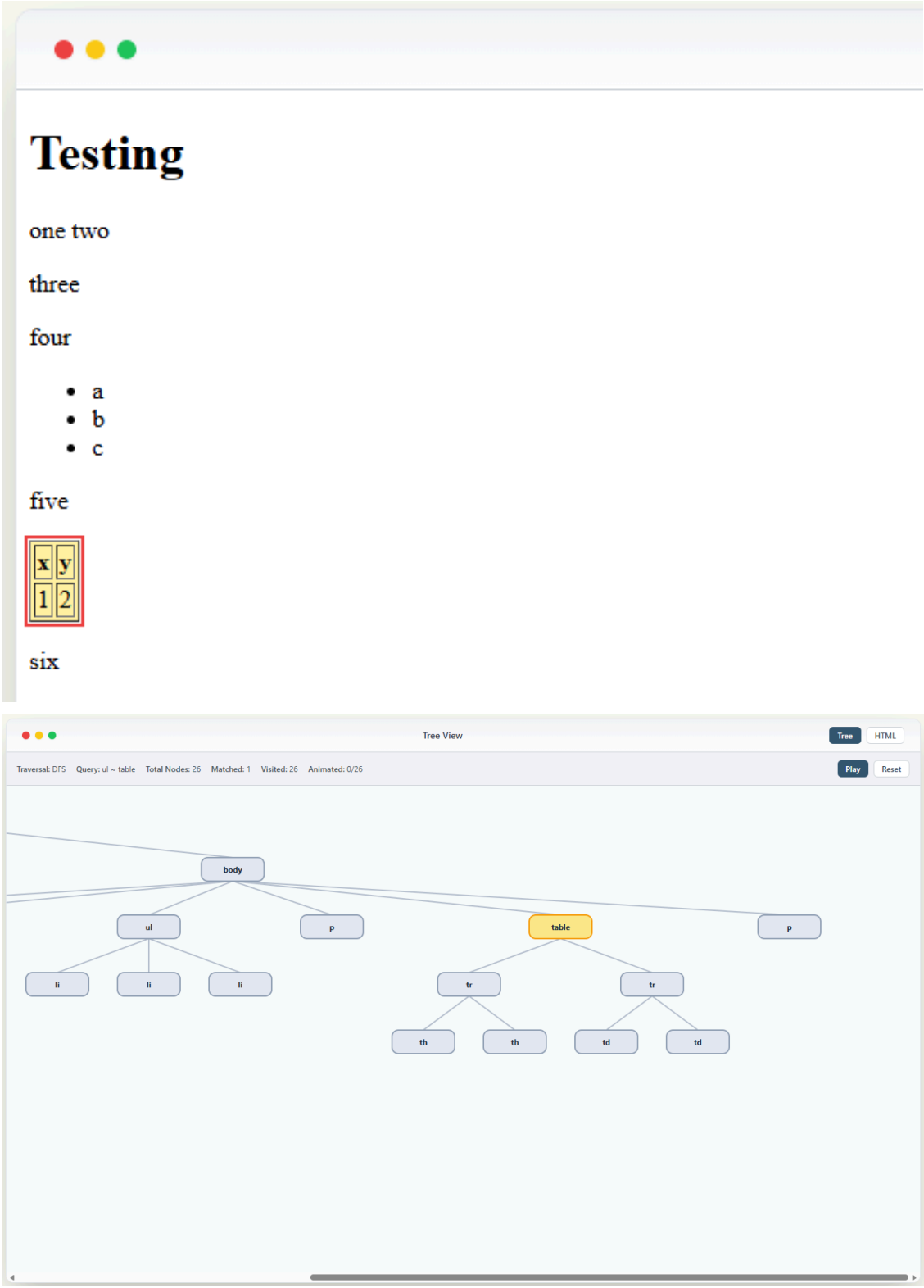

Tree View
Tree
HTML
Traversal: DFS
Query: div p
Total Nodes: 26
Matched: 3
Visited: 26
Animated: 0/26
Play
Reset
html
head
body
meta
h1
div
ul
p
meta
p
p
section
li
li
li
tr
title
span
p
th

ul + p


HTML View
Testing
one two
three
four
a
b
c
five
x y
1 2
six

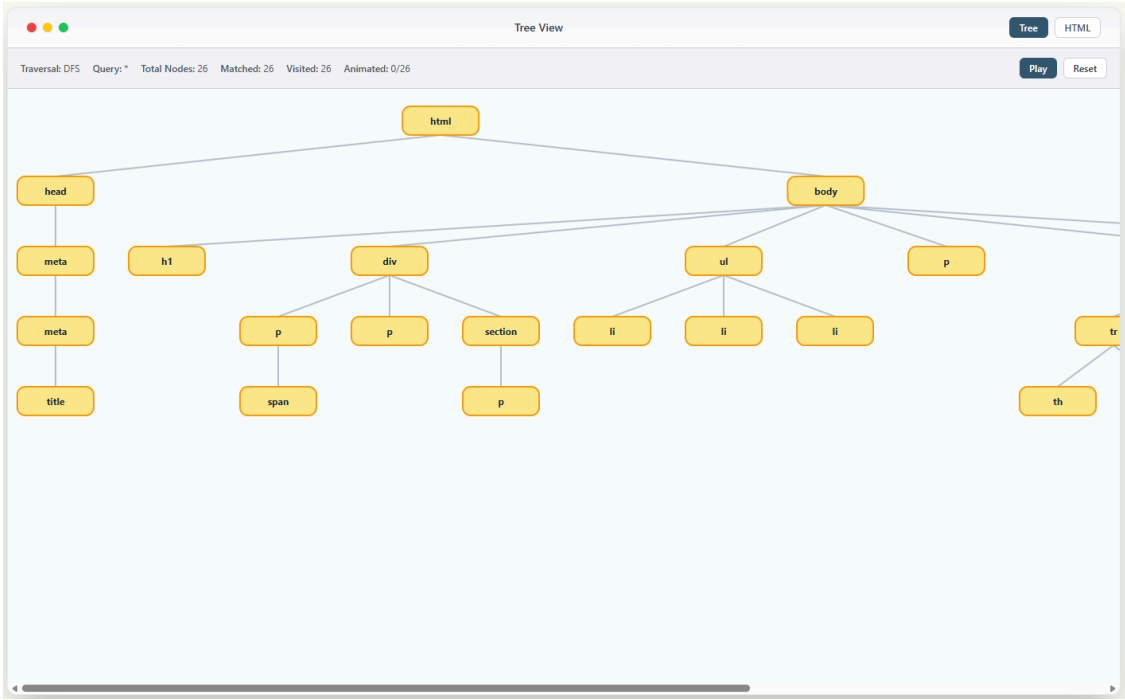

Tree View
Tree
HTML
Traversal: DFS
Query: ul ~ p
Total Nodes: 26
Matched: 1
Visited: 26
Animated: 0/26
Play
Reset
html
body
div
ul
p
table
p
section
li
li
li
tr
tr
p
th
th
td
td

ul ~ table

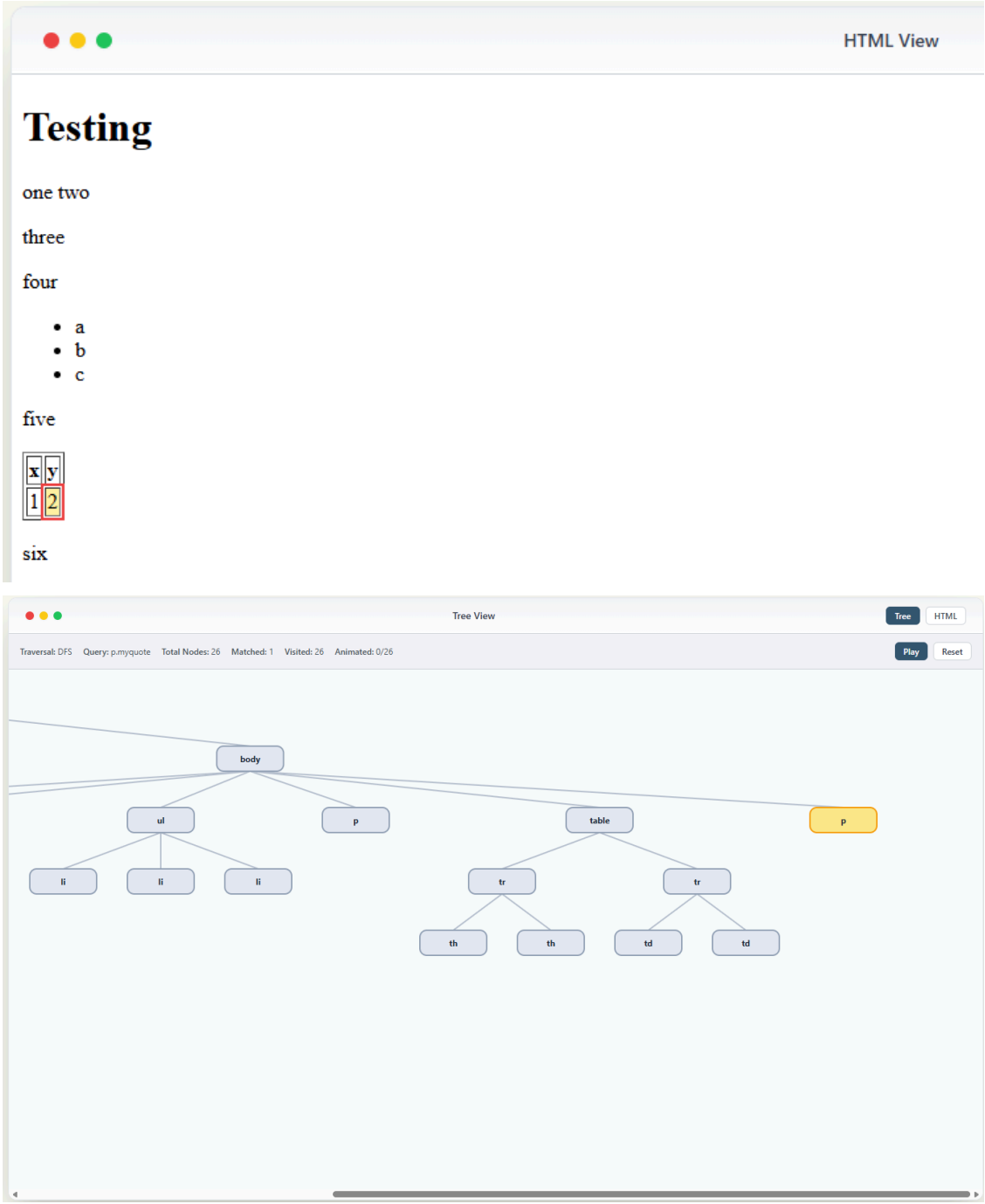

Testing
one two
three
four
a
b
c
five
x y
1 2
six
Tree View
Tree
HTML
Traversal: DFS
Query: ul ~ table
Total Nodes: 26
Matched: 1
Visited: 26
Animated: 0/26
Play
Reset
body
ul
p
table
p
li
li
li
tr
tr
th
th
td
td

*

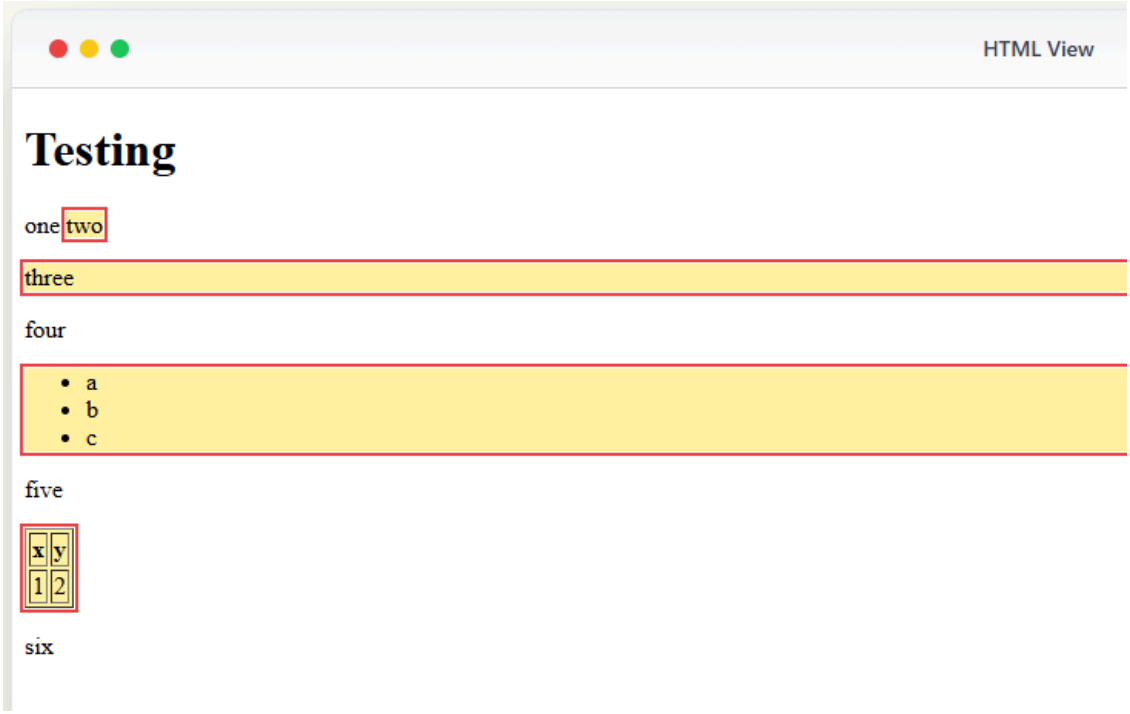
HTML View
Tree
HTML
Testing
one two
three
four
a
b
c
five
x y
1 2
six

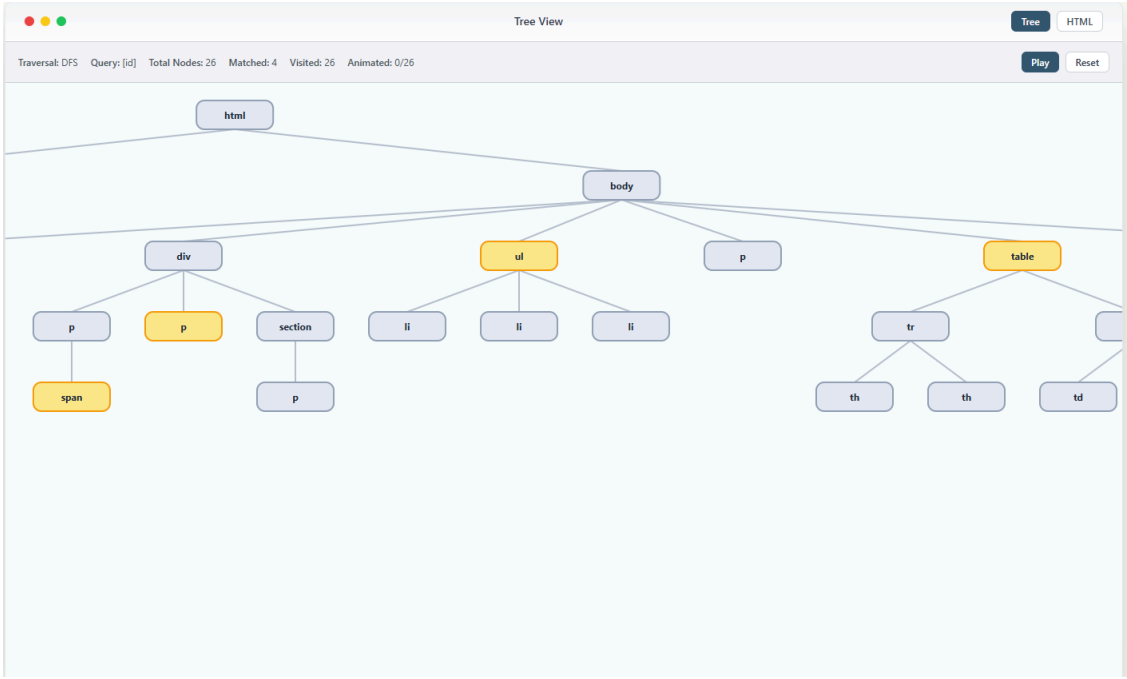

Tree View
Tree
HTML
Traversal: DFS
Query: *
Total Nodes: 26
Matched: 26
Visited: 26
Animated: 0/26
Play
Reset
html
head
body
meta
h1
div
ul
p
meta
p
p
section
li
li
li
tr
title
span
p
th

p.myquote

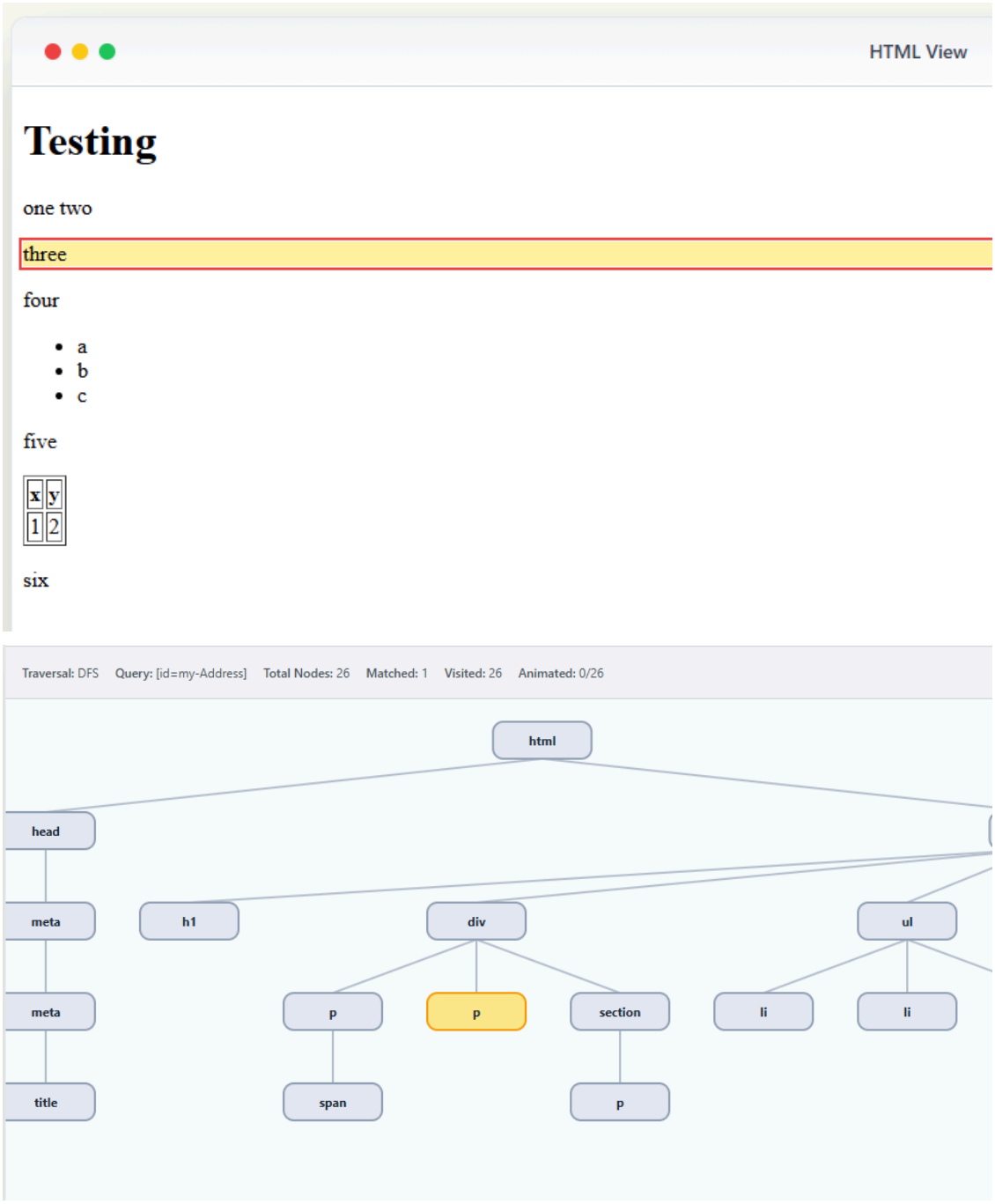

HTML View
Testing
one two
three
four
a
b
c
five
x y
1 2
six
Tree View
Tree
HTML
Traversal: DFS
Query: p.myquote
Total Nodes: 26
Matched: 1
Visited: 26
Animated: 0/26
Play
Reset
body
ul
p
table
p
li
li
li
tr
tr
th
th
td
td

[id]

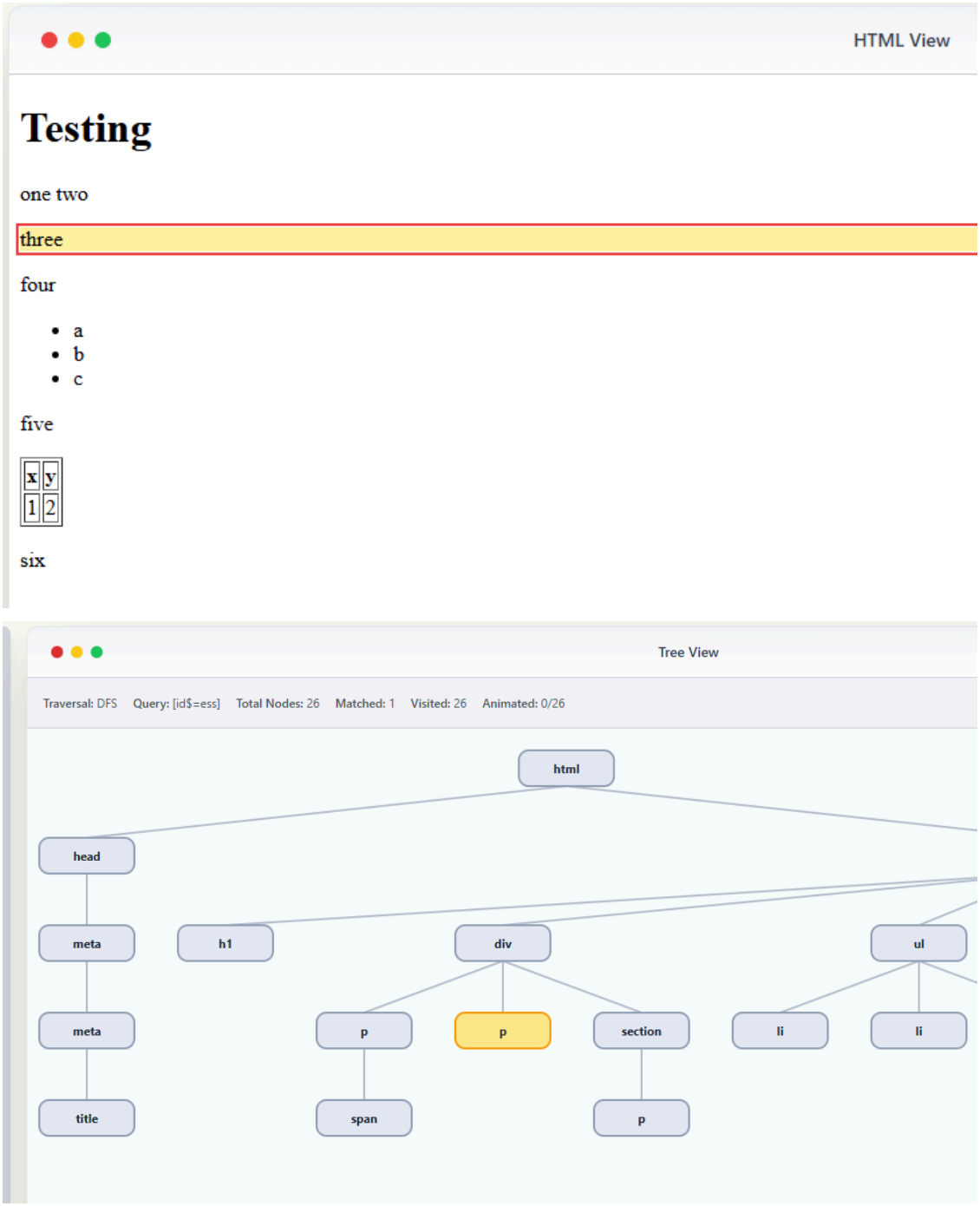




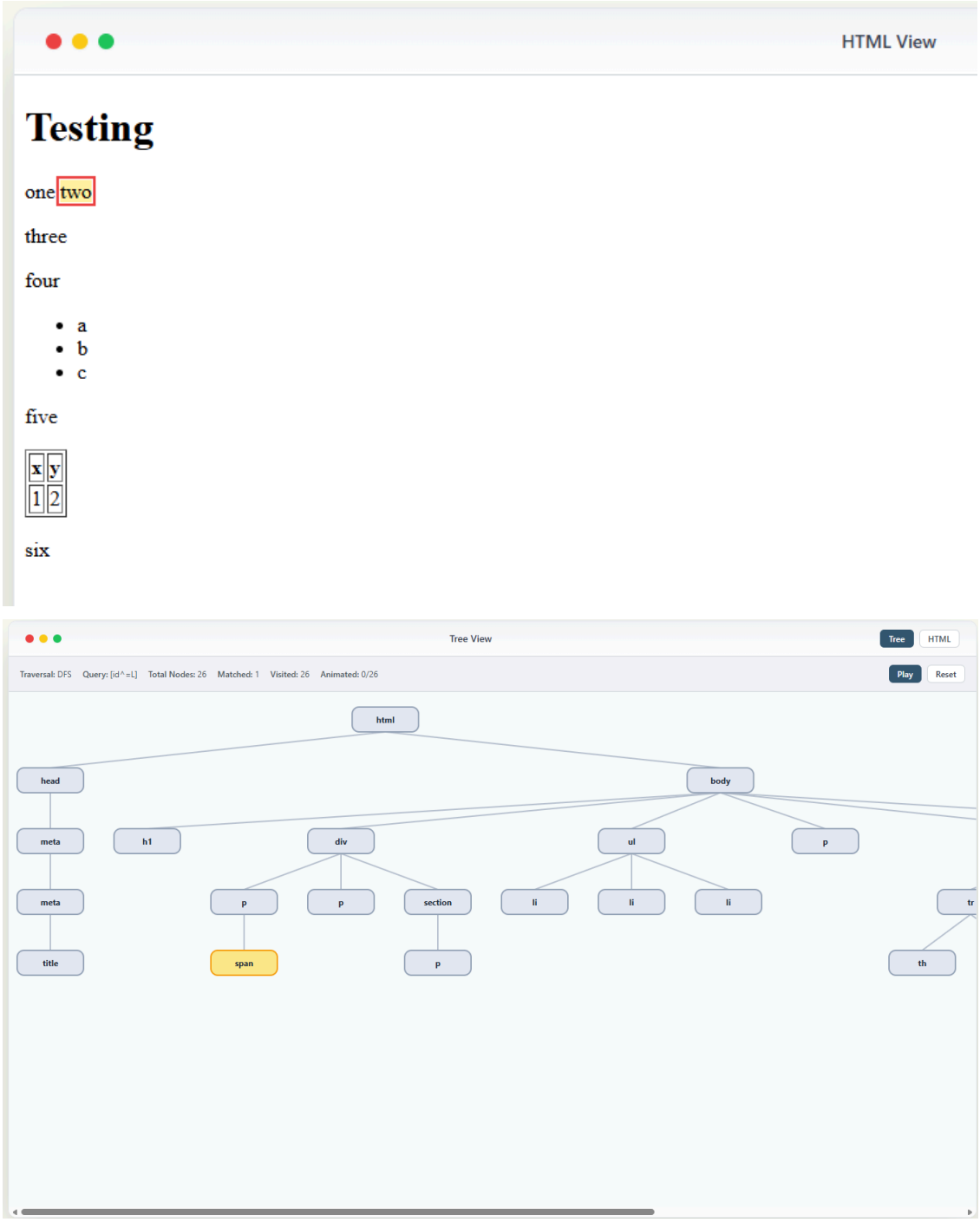
[id=my-Address]


HTML View
Testing
one two
three
four
a
b
c
five
x y
1 2
six
Traversal: DFS Query: [id=my-Address] Total Nodes: 26 Matched: 1 Visited: 26 Animated: 0/26
html
head
meta
meta
title
h1
div
p
p
span
section
p
ul
li
li

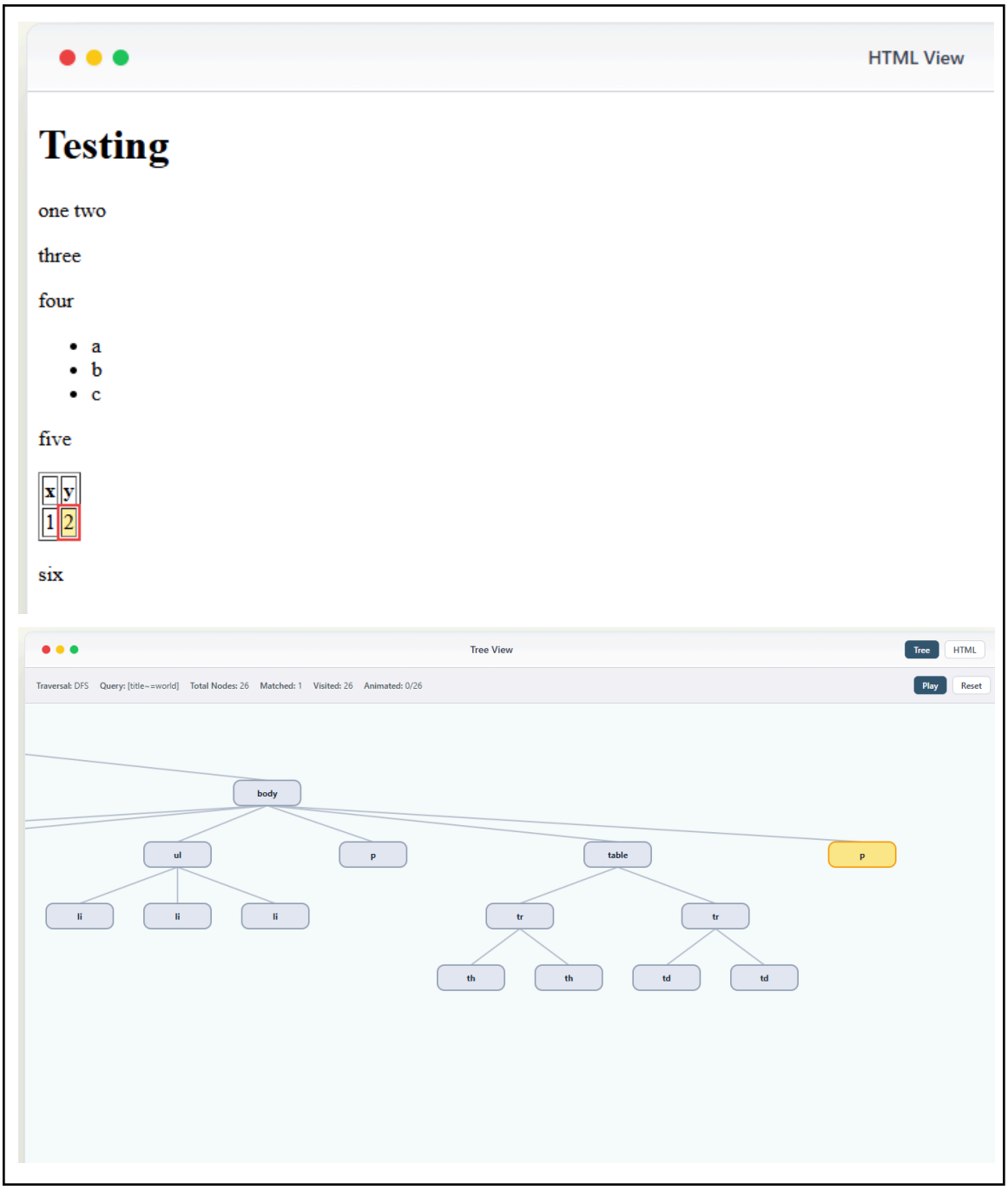
[id$=ess]


HTML View
Testing
one two
three
four
a
b
c
five
x y
1 2
six
Tree View
Traversal: DFS Query: [id$=ess] Total Nodes: 26 Matched: 1 Visited: 26 Animated: 0/26
html
head
meta
h1
div
ul
meta
p
p
section
li
li
title
span
p

[id^=L]


HTML View
Testing
one two
three
four
a
b
c
five
x y
1 2
six
Tree View
Tree
HTML
Traversal: DFS
Query: [id^=L]
Total Nodes: 26
Matched: 1
Visited: 26
Animated: 0/26
Play
Reset
html
head
body
meta
h1
div
ul
p
meta
p
p
section
li
li
li
tr
title
span
p
th

[title~=world]


HTML View
Testing
one two
three
four
a
b
c
five
x y
1 2
six
Tree View
Tree
HTML
Traversal: DFS
Query: [title~=world]
Total Nodes: 26
Matched: 1
Visited: 26
Animated: 0/26
Play
Reset
body
ul
p
table
p
li
li
li
tr
tr
th
th
td
td

5. BAB 5 Kesimpulan

5.1. Kesimpulan

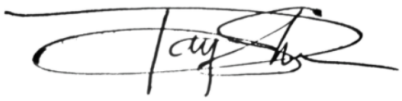
Pengembangan sistem ini telah berhasil merealisasikan fungsionalitas penelusuran dan pencarian elemen secara spesifik pada struktur *Document Object Model* (DOM) *Tree*. Sistem mampu mengevaluasi parameter masukan berupa *CSS Selector* dan memetakannya ke dalam bentuk representasi graf hierarkis dan melakukan visualisasi, animasi, melakukan penampilan dan *parsing* dari input link html dengan *scraping.*

6. Lampiran

Anggota	Pekerjaan
Billie Bhaskara Wibawa	Frontend Backend Integration, Unit Testing, Deployment, Multithreading, Parser, Web Crawler, Docker, Laporan
Raysha Erviandika Putra	BFS, DFS, CSS Selector, Testing, Laporan, Frontend Animasi-HTML
Dzaki Ahmad Al Hussainy	Early Planning, Front End Overall UI, UI/UX, Video, Laporan, README

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Billie Bhaskara Wibawa



Raysha Erviandika Putra

Dzaki Ahmad Al Hussainy

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video	✓	
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading	✓	
13	[Bonus] Implementasi LCA Binary Lifting		✓

Github: billie-bytes/PartlyOtomata: HTML parser and CSS selector
Youtube: https://youtu.be/1eq0uqb6iz8
Deployment: https://otomataparty-beb5f0esajgbbbd4.eastasia-01.azurewebsites.net/

7. Daftar Pustaka

W3school. (n.d.). T*ry CSS Selector*. Diakses pada 25 April 2026, Try CSS Selector

GeeksforGeeks. (n.d.). *Difference between BFS and DFS*. GeeksforGeeks. Diakses pada 25 April 2026, dari Difference between BFS and DFS - GeeksforGeeks